

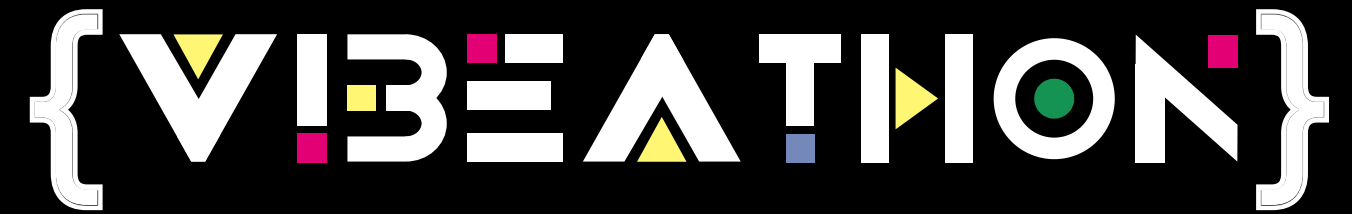
Data

Problem Statement & Use Case

- Across ITDL products (App, Shop, Bot, TV, Broadband), event tagging is inconsistently applied across journeys and platforms.
- Tagging is manually maintained and prone to drift over time.
- No real-time validation when new features or releases go live.
- Analytics reliability, cross-journey comparability, and time-to-insight are all impacted.
- AI-powered automated event glossary generation & maintenance from existing event streams, schemas, and historical data — continuously updated as new features and journeys are introduced.
- Event standardisation & prediction engine to detect duplicates, suggest/auto-correct names and properties, and predict missing or incorrectly tagged events in new releases.
- Stretch: regression/validation suite that flags missing/non-compliant tags and integrates into deployment pipelines.

Expected Outcome

- Scalable, AI-driven tagging governance system.
- Improved consistency across App, Shop, Bot, TV journeys.
- Reduced manual effort in tagging design and validation.
- Faster and more reliable analytics readiness for new features.



SEN

Problem Statement & Use Case

- Digital penetration is low; app experience needs strengthening.
- One user may have multiple party IDs, each with multiple profiles — no unified identity across login methods.
- Push-eligible base is only 40%; sessions driven by push notifications are at 1% per month in TDG.
- Login success rates are low; 2FA is complex and adoption is poor.
- App start time is slow; splash page experience needs improvement.
- Strong app experience with world-class home page personalisation.
- High-frequency use cases to drive MoM retention and stickiness.
- High-intent fulfilment from OA home page.
- Unified user profile — 1 user = 1 profile regardless of login method.
- Push notifications reaching all permissioned users with improved CTR and engagement.
- Simplified 2FA driving higher login success and adoption.

Expected Outcome

- 99.9%+ CFR (QOS)
- <5% home page bounce rate
- Push-eligible base grows from 40% to 80%+
- % sessions from push notifications grows from 1% to 8%+ in TDG
- Blazing-fast app start time
- Improved MoM retention



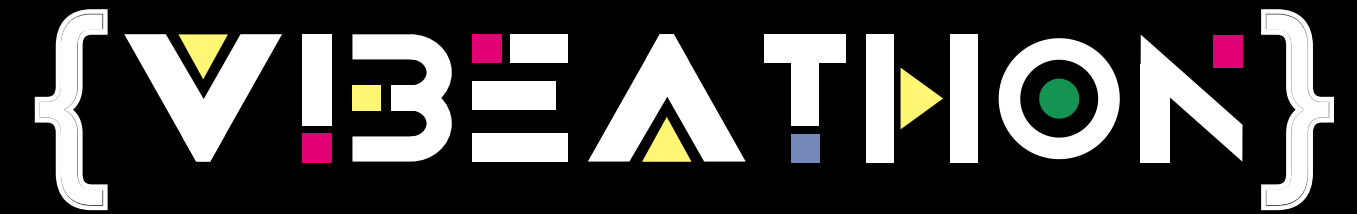
Home (TV)

Problem Statement & Use Case

- With our far-field enabled TV hardware and the power of advanced Voice + AI, reimagine a smarter, more intuitive TV experience.

Expected Outcome

Reimagine a smarter, more intuitive TV experience.



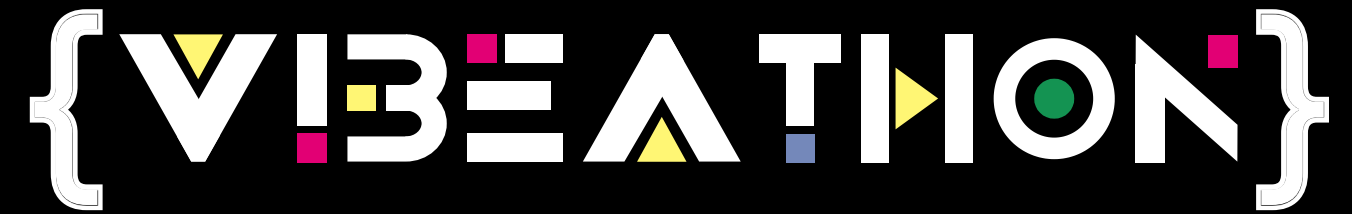
OneAI

Problem Statement & Use Case

- ThinkFlow: Build Your Own AI Problem-Solving Agent.
- Enable every employee to create intelligent, task-oriented agents using simple natural-language instructions.
- Agents should understand a goal, break it into logical steps, reason over information, and use relevant tools or enterprise data to produce a meaningful outcome through a workflow-based experience.
- Example: a user could create an Analytical Agent that gathers data, identifies trends, performs calculations, generates insights, and presents the result in a business-friendly format, without requiring advanced programming skills.
- Participants are expected to explore open agentic frameworks such as LangGraph and LangChain Deep Agents to build modular, reasoning-capable agents.

Expected Outcome

- A workflow-based platform for every employee to build their own reasoning-capable AI agents, no coding required.
- Agents that take a stated goal, reason over information and enterprise data, and produce a business-ready outcome.
- Reusable building blocks, like an Analytical Agent, that support diverse business scenarios across the organisation.
- Faster, more accessible problem-solving using open agentic frameworks such as LangGraph and LangChain Deep Agents.



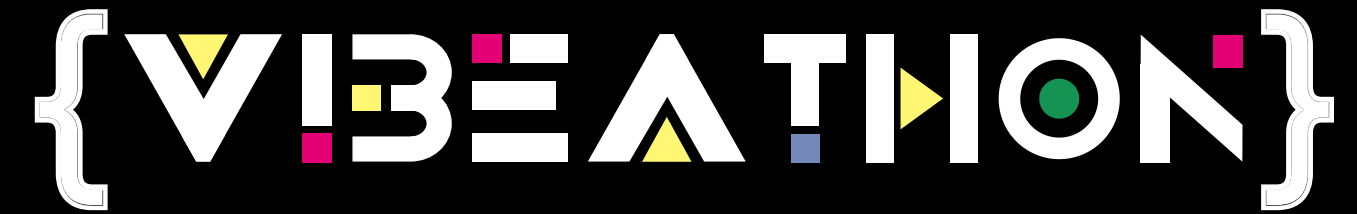
OneAI

Problem Statement & Use Case

- Voice customer support transcripts contain rich conversational intelligence — query types, phrasing styles, agent response patterns, clarification techniques - but this data is buried in unstructured text, not standardised, and difficult to analyse at scale.
- AI voice systems fail to replicate effective human-like support interactions because conversational patterns are never extracted or modelled.
- Customer-side gap: no system captures how users describe problems verbally, what kinds of questions are asked (informational, troubleshooting, confirmation), or recurring phrasing patterns in spoken language.
- Agent-side gap: no system captures which response styles (instructional, empathetic, diagnostic) lead to faster resolution, or how agents ask follow-up and clarifying questions naturally.
- Interaction-flow gap: common multi-turn conversational patterns, typical back-and-forth sequences, and drop-offs or friction points in verbal communication are invisible and unmodelled.
- Build a modular NLP platform that processes Sprinklr voice transcripts, maps conversations to issue taxonomy, and extracts question types, intent phrasing, agent response patterns, and multi-turn interaction flows.
- Cluster conversations by style and intent using embedding-based techniques and generate structured representations of conversational patterns.
- Voice-first focus throughout — prioritising natural conversational cadence over text-based interaction patterns
- Platform must be modular and plug directly into existing voice bot infrastructure (e.g. via LiveKit), applying learned patterns in real-time by issue type.

Expected Outcome

- The system should cluster conversations by intent, issue type, and conversational style.
- It should generate structured insights on customer phrasing, question types, agent response patterns, and effective resolution flows.
- A working modular platform that can integrate with existing voice bot agentic system.
- The platform should help voice bots respond more naturally, empathetically, and accurately in real-time interactions.
- Success must be measured through evaluation metrics such as improved engagement score, higher intent accuracy, and reduced unresolved conversations due to misclassification.



Architecture

Problem Statement & Use Case

- OneArch holds 6+ years of architecture knowledge (the OneArch Golden Data Set), currently used by all architects in day-to-day work via Cursor AI, but there are no dedicated EA or SA agents integrated into AIDLC.
- Architecture knowledge base is not automatically updated when MRs are approved and deployed.
- Build Enterprise Architect & Solution Architect agents integrated with AIDLC to produce accurate solution analysis and MAPI examples for engineering agents.
- Automate architecture knowledge base updates on every approved and deployed MR.
- Prove the Living Stack model through one high-impact internal use case.

Expected Outcome

- 1 working internal AI solution with real users and real impact (within 180 days).
- 1 reusable delivery model — the Living Stack blueprint.
- 1 scaling decision — where to roll out next.



Home (Broadband)

Problem Statement & Use Case

CPE-related issues — WiFi, core router, voice, platform, telemetry — are typically surfaced by the customer rather than the network. Detection is reactive, diagnosis depends on manual investigation, and remediation waits on human intervention. The result is longer time to resolution, avoidable support contacts, and degraded service that persists until someone complains.

Use case:

- Build Agentic AI subject matter experts that autonomously detect, triage, diagnose and remediate CPE issues. Pick one of the two below, or both.
- External Agentic AI SME — an agent operating outside the internal firmware stack, observing the gateway through external data streams. Choose one domain: WiFi, Core Router, Voice, Platform or Telemetry. Multiple agents are permitted, but depth in a single domain scores higher than breadth.
- Edge AI Agents — lightweight agents running at or near the CPE, continuously analysing local telemetry, events and device/network state.

Expected Outcome

External Agentic AI SME

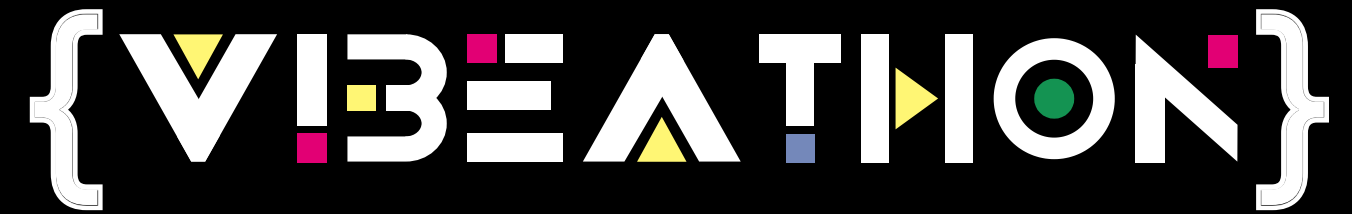
- A novel, relevant idea with demonstrable impact on CPE issue resolution
- Demonstrated domain depth — the approach and data sources used to build CPE expertise
- A working agent — architecture, reasoning approach, technologies, tools, and integration with external CPE data sources
- Root cause identification and self-healing — appropriate remediation actions, executed or recommended, with reasoning a human can follow

Edge AI Agents

- A novel, relevant idea for applying AI at or near the CPE
- Local detection, diagnosis and remediation — the approach and data sources behind edge intelligence
- A working edge agent — architecture, AI/ML models, technologies executing at the edge
- Proven feasibility in constrained environments — CPU, memory, storage, resource efficiency, response latency, connectivity dependency
- Safe autonomous remediation — effective self-healing that doesn't risk service

Both

- All of the above, plus
- End-to-end architecture — effective collaboration between edge and external agents: context exchange, distribution of intelligence and reasoning, coordination of diagnosis and remediation.



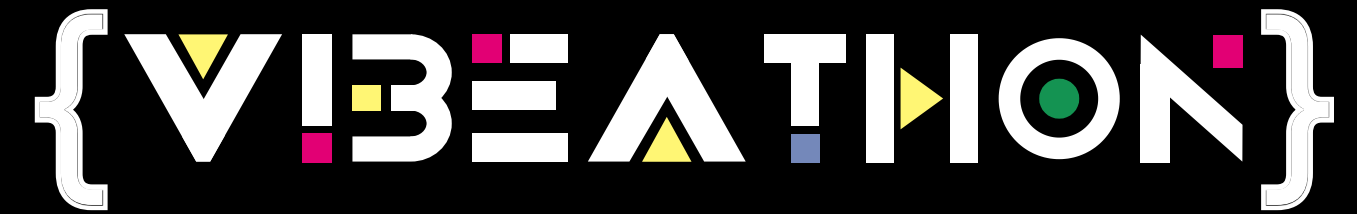
Commerce

Problem Statement & Use Case

- When a stability incident occurs (latency spike, error rate jump, or client crash), engineers must manually correlate signals across four layers: Kubernetes pod logs, BFF metrics, CDN/web error tracking, and mobile crash traces.
- No single system correlates backend JVM thread pool exhaustion with a BFF timeout and the resulting native app crash or web error spike.
- Each incident is triaged from scratch — correlation logic and past resolutions live in individual engineers' heads or scattered Slack threads.
- Context-switching turns a minutes-long root-cause into a multi-hour war room.
- Build an Autonomous Stability Agent (multi-agent) that observes real-time signals from all four layers: web app, native app, BFF, and Spring Boot backend.
- Auto-correlate failures — e.g. backend DB connection pool exhaustion → BFF latency degradation → spike in mobile app errors.
- Query past incidents via a knowledge graph/vector memory to surface similar historical patterns and resolutions.
- Recommend safe remediations (cache invalidation, circuit breaker tuning, pod restart) with human-in-the-loop approval for destructive actions.

Expected Outcome

- Detect a synthetic cross-layer failure (e.g. backend slowdown → BFF timeout → client error rate jump) within 2 minutes of the first alert.
- Assemble a correlated root-cause hypothesis without human log-jumping.



Fintech

Problem Statement & Use Case

- Configuring a new product launch in OneX currently involves multiple integration activities, stretching timelines to months.
- Examples: launch of cyber insurance with mobile contract, selling home insurance as a new contract, adding travel insurance as an add-on.

Expected Outcome

- New product configured and launched in OneX within a week instead of months.